



SE618 LOGISTIC CASE STUDY ANALYSIS

WEEK 10 – PROGRESS REPORT

PROJECT TITLE: SERGE – AI Agent for Supply Chain Operations

Code Hardening, Data Model Refinement & Desktop Integration

Gauthier SCHMITT	6822807466
Louis GRIGNOLA	6822807516
Matteo SCHWARZ	6822807557
Maël TELLUS	6822807532

SUBMITTED TO:

AJARN WARUT

APRIL 2026

SEMESTER 2/2025

**SCHOOL OF MANUFACTURING SYSTEMS AND MECHANICAL ENGINEERING
SIRINDHORN INTERNATIONAL INSTITUTE OF TECHNOLOGY
THAMMASAT UNIVERSITY**

Contents

1	Summary	2
2	Code Hardening & Data Model Refinement	2
2.1	What We Did	2
2.2	Data Model Fixes	2
2.3	Code Quality Improvements	2
3	MCP Integration & Desktop UI	3
3.1	The Problem	3
3.2	How MCP Makes This Easy	3
3.3	Desktop Integration	3
3.4	What the User Sees	4
4	Current State of the Project	4

Summary

This week focused on two goals: **stabilising the codebase** and making SERGE **easy to use for anyone**. We went through the entire Go backend to catch edge cases and tighten the data model, and we reworked the MCP integration so that SERGE can be plugged into any desktop agent client in a few clicks.

Code Hardening & Data Model Refinement

What We Did

After the first round of agent testing (Week 8), we noticed a few areas where the code and data were fragile. This week we went through the full stack — domain models, repositories, services, controllers, and seed data — to fix issues and improve robustness.

Data Model Fixes

Table 1: Data model issues identified and resolved.

Issue	Resolution
Inconsistent stock movement references	Standardised <code>reference_type</code> values to <code>supplier_order</code> and <code>customer_order</code> across all data files and the seed generator.
Missing supplier product links	Verified every <code>SupplierOrderLine.supplier_product_id</code> points to a valid <code>SupplierProduct</code> , and every <code>SupplierProduct.product_id</code> maps to an existing <code>Product</code> .
Scenario data integrity	Ran a verification script against all 15 evaluation scenarios to confirm ground truth values match the actual seed data (ORD-007 status, PROD-001 inventory, PO-008 details, SHP-009 exception).
Seed generator idempotency	The seed generator (<code>cmd/seedgen</code>) now produces identical output on every run (fixed random seed = 42), making results fully reproducible.

Code Quality Improvements

- **Seed loader refactored** using Go generics — one `loadEntities[T]` function replaces 12 repetitive load blocks, reducing the loader from 90 lines to 40.
- **Repository loaders** simplified to one-liner methods for consistency.
- **Main server startup** condensed — repository, service, and controller creation now fits in a single readable block.
- **Dead code removed** — old `GetShipmentByOrder` (single return) replaced by `ListShipmentsByCustomerOrder` (supports split shipments).

Key Insight

These changes are not visible to the end user, but they matter for the project’s credibility. A clean, well-tested codebase shows that we understand software engineering beyond just “making it work.” The seed generator’s reproducibility is particularly important for evaluation: anyone can regenerate the exact same 500-order dataset and verify our results.

MCP Integration & Desktop UI

The Problem

Until now, using SERGE required command-line knowledge: building the Go binary, configuring `.mcp.json`, and running Claude Code in a terminal. This is fine for developers but not for the operations managers who are our target users.

How MCP Makes This Easy

The Model Context Protocol was designed to be a universal plug — like USB-C for AI agents. Our MCP server communicates over **stdio** (standard input/output), which means any MCP-compatible client can launch it as a subprocess and immediately access all 35 supply chain tools.

Desktop Integration

SERGE is now pluggable into any MCP-compatible desktop agent in a few steps:

1. **Build the binary:** one command (`go build -o serge-mcp cmd/server/main.go`).
2. **Add to your agent client:** point the client to the binary and the data folder. For Claude Desktop, this means adding a few lines to the MCP configuration file. For other clients (Cursor, Windsurf, or any MCP-compatible tool), the process is similar.
3. **Start using it:** the agent immediately sees all 35 tools and can answer supply chain questions.

Table 2: SERGE integration with different agent clients.

Client	Setup	Notes
Claude Code (CLI)	<code>.mcp.json</code> in project folder	Already working since Week 8
Claude Desktop (Mac)	Add to MCP settings	Full GUI, no terminal needed
Cursor / Windsurf	MCP server config	IDE-integrated agent
Custom agent	Connect via stdio	Any MCP SDK (Python, TS, Go)

Key Insight

This is a key result of the project: SERGE is **agent-agnostic**. We built the supply chain infrastructure once, and any AI agent can plug into it. Switching from Claude to GPT or Llama requires zero changes to our code — only the agent client changes. This validates our architecture choice of using MCP as the integration layer.

What the User Sees

From the user’s perspective, the experience is simple: open their desktop agent (e.g., Claude Desktop), and ask questions in natural language. The agent calls SERGE’s MCP tools in the background and returns a structured answer. The user never interacts with the supply chain systems directly.

Example interaction:

User: “Why is order ORD-007 stuck?”

Agent: Checks OMS (order stuck in processing, needs 50 units of Nam Doc Mai), checks TMS (no shipment created), checks WMS (only 8 units in stock), checks SRM (PO-008 incoming with 150 units, arriving Feb 18). Returns a full diagnostic with root cause and timeline.

Time: <60 seconds vs. 15–30 minutes manually.

Current State of the Project

Table 3: Project deliverables and status.

Deliverable	Status	Details
Go backend (4 domains)	Done	Clean architecture, 35 MCP tools, all compiling
Seed data (realistic)	Done	500 orders, 60 POs, 493 shipments, reproducible
Seed generator	Done	Idempotent, fixed seed, scenario data preserved
MCP integration	Done	Stdio transport, works with any MCP client
Desktop integration	Done	Tested with Claude Code and Claude Desktop
Business context (CLAUDE.md)	Done	Enum definitions, business rules, reasoning patterns
Evaluation scenarios	Done	15 scenarios with ground truth
Agent testing (qualitative)	Done	4 queries tested, scores 4.7–5.0/5.0
LLM-as-Judge pipeline	In progress	Judge prompt and scoring rubric being finalised
Final report	Planned	Week 11